



Agentic AI Developer Case Study – Enterprise Knowledge Ops Agent

1. Problem Statement

Enterprises today manage **large and diverse sets of unstructured documents**—such as policies, Standard Operating Procedures (SOPs), contracts, compliance manuals, and audit reports. Business users often need **accurate, explainable answers** that require reasoning across multiple documents rather than simple keyword-based retrieval.

Most traditional Retrieval-Augmented Generation (RAG) chatbots are limited in the following ways:

- They respond **reactively** without planning multi-step reasoning.
- They lack **specialized roles** (for retrieval, reasoning, validation, and memory).
- They provide little visibility into **how or why** an answer was generated.

This creates a gap between what enterprises need and what basic LLM-powered systems can deliver.

Learner Challenge

In this case study, learners take on the role of an **Augmented Engineering Lead** responsible for designing and building an **Enterprise Knowledge Operations Agent**. The goal is to move beyond a single “chatbot” and instead construct a **coordinated, multi-agent AI system** that can:

- **Plan** how to answer complex business questions
- **Delegate tasks** to specialized agents

Focus

ENG
IN



Agentic AI Developer Case Study – Enterprise Knowledge Ops Agent

1. Problem Statement

Enterprises today manage **large and diverse sets of unstructured documents**—such as policies, Standard Operating Procedures (SOPs), contracts, compliance manuals, and audit reports. Business users often need **accurate, explainable answers** that require reasoning across multiple documents rather than simple keyword-based retrieval.

Most traditional Retrieval-Augmented Generation (RAG) chatbots are limited in the following ways:

- They respond **reactively** without planning multi-step reasoning.
- They lack **specialized roles** (for retrieval, reasoning, validation, and memory).
- They provide little visibility into **how or why** an answer was generated.

This creates a gap between what enterprises need and what basic LLM-powered systems can deliver.

Learner Challenge

In this case study, learners take on the role of an **Augmented Engineering Lead** responsible for designing and building an **Enterprise Knowledge Operations Agent**. The goal is to move beyond a single “chatbot” and instead construct a **coordinated, multi-agent AI system** that can:

- **Plan** how to answer complex business questions
- **Delegate tasks** to specialized agents
- **Reason across multiple documents**
- **Validate and verify** responses before returning them
- **Explain decisions and sources** clearly to end users

This mirrors real-world enterprise AI development challenges, where **accuracy, trust, and explainability** are as important as raw model capability.

2. Scope of the System

The scope of this case study is to design and build a **local, agentic AI system** that can answer complex enterprise questions by reasoning across multiple documents using a **multi-agent architecture**.

In Scope

Learners are expected to build a system that:

- **Ingests enterprise documents** (e.g., policies, SOPs, contracts) and stores them in a vector database for retrieval
- **Implements a multi-agent design**, where each agent has a clearly defined role such as planning, retrieval, reasoning, validation, and memory management



ed to edit, it's safer to stay in Protected View.

Enable Editing



Cognizant Skillspring

- **Handles complex user queries** that require multi-step reasoning and cross-document synthesis rather than single-pass answers
- **Plans and orchestrates tasks** by decomposing a query and routing subtasks to appropriate agents
- **Validates and grounds responses** against source documents before presenting the final answer
- **Demonstrates AI guardrails**, including hallucination control, input validation, and clear source attribution
- **Runs fully in a local environment**, focusing on correctness, reasoning, and governance rather than production scalability

Out of Scope

- Cloud or production deployment
- Enterprise authentication or access control
- Model training or fine-tuning
- Advanced UI or frontend optimization

3. Skill Tower (Pick any 1 Skill tower – Open Source, Azure, AWS, GCP)

Skill Names (Associates can choose technology as per their preference, which is available in the syllabus)
Note: Below is for Open Source Frameworks
• Python • LangChain • LangGraph • Embeddings • RAG (Retrieval Augmented Generation) • Azure OpenAI • Vector database – Flass / Pinecone / Chroma • AI Guardrails & Governance • Deployment to AWS • Optional UI (Streamlit / Chainlit / Front End Technology of your choice)

4. User Stories

User Story 1: Complex Query Handling

As a business user, I want to ask complex questions across multiple enterprise documents to receive accurate, well-reasoned, and explainable answers rather than single-pass responses.

Acceptance Criteria

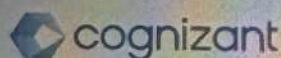
- The system accepts queries that require reasoning across more than one document
- The query is decomposed into logical subtasks by an orchestrator agent
- Relevant documents are retrieved and used to synthesize a final answer
- The final response is coherent and clearly answers the original question

User Story 2: Agent Planning and Orchestration

As a developer, I want the system to plan and route tasks to specialized agents so that each step of reasoning is handled by the most appropriate agent.

to edit, it's safer to stay in Protected View.

Enable Editing



Cognizant Skillspring

Acceptance Criteria

- A planning or orchestrator agent determines the sequence of actions needed
- Distinct agents are invoked for retrieval, reasoning, validation, and memory
- Agent interactions are traceable and logged for inspection

User Story 3: Grounded and Validated Responses

As a business user, I want confidence that responses are grounded in source documents so that I can trust the system's output.

Acceptance Criteria

- Retrieved sources are explicitly linked to the generated answer
- A verifier or validation agent checks grounding before final output
- If grounding confidence is low, the system flags or limits the response

User Story 4: Explainability and Transparency

As a learner or reviewer, I want visibility into how the system arrived at an answer so that I can understand and evaluate agentic decision-making

Acceptance Criteria

- The system exposes agent decision traces or reasoning summaries
- Retrieval results and validation outcomes are logged and viewable
- The explanation aligns with the final response and source documents

User Story 5: Governance and Guardrails

As an enterprise stakeholder, I want guardrails in place so that the system minimizes hallucinations and handles uncertainty responsibly.

Acceptance Criteria

- Input validation and basic safety checks are implemented
- The system applies hallucination controls or confidence thresholds
- Clear source attribution or disclaimers are provided in responses

User Story 6: Evaluation, Observability, and Failure Detection

As a learner or system evaluator, I want the system to implement explicit evaluation mechanisms—such as grounding checks, retrieval relevance scoring, agent decision traces, and failure detection—so that I can assess the reliability, transparency, and correctness of agentic responses.

Acceptance Criteria

- The system performs **grounding checks** to verify that generated responses are supported by retrieved source documents
- **Retrieval relevance scores** or signals are captured for the documents used in answering a query

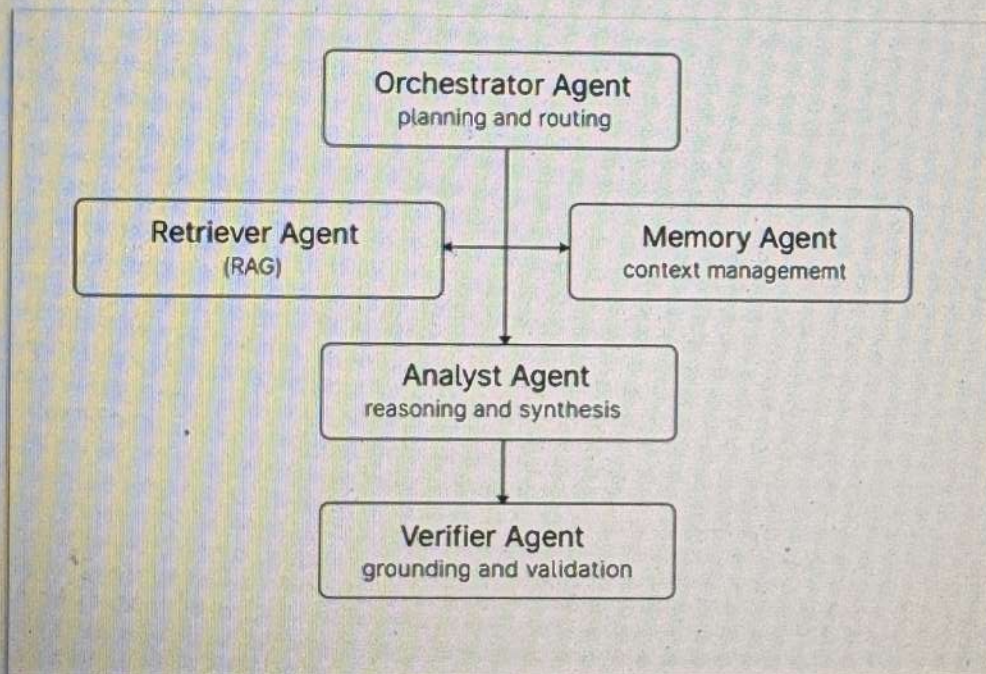


Cognizant Skillspring

- **Agent decision traces** (e.g., planning steps, agent invocation order, key decisions) are logged and accessible
- The system can **detect and flag failures**, such as:
 - Insufficient or irrelevant retrieval
 - Low grounding confidence
 - Conflicting agent outputs
- **Evaluation outputs are logged** in a structured and inspectable format (e.g., console logs, files, or UI view)
- The system provides a **clear explanation** of evaluation results alongside or in support of the final response

5. Agent Architecture

(Guidance only)



6. Expected Deliverables

- Application code base.
- Fully functional application that should be running and working.
- Architecture and agent flow diagram
- Evaluation and guardrail documentation
- Unit test case documentation

7. Evaluation Rubric

This rubric explains **how learner submissions will be evaluated** and what constitutes a **strong, average, or weak implementation** of an Agentic AI system.

Focus





Overall Scoring Guidance

Cognizant Skillspring

- Total Evaluation: 100 points
- Emphasis is on **agentic reasoning, validation, evaluation, and governance**, not UI polish or cloud deployment

1. Agentic Architecture & Design (20 points)

What learners are evaluated on:

How well the system demonstrates a **true multi-agent architecture** with clear responsibilities.

Level	Indicators
Excellent (16–20)	Clearly defined agents (Orchestrator, Retriever, Analyst, Verifier, Memory) with distinct roles; orchestration logic is explicit and well-documented
Good (11–15)	Multiple agents implemented, but responsibilities partially overlap or orchestration is implicit
Needs Improvement (0–10)	Single-agent or loosely structured system presented as “agentic”

2. Query Planning & Orchestration (15 points)

What learners are evaluated on:

Ability to decompose a user query and route tasks across agents.

Level	Indicators
Excellent (12–15)	Orchestrator plans multi-step execution and routes subtasks logically
Good (8–11)	Basic routing present but limited planning depth
Needs Improvement (0–7)	No visible planning; linear execution only

3. Retrieval & RAG Effectiveness (15 points)

What learners are evaluated on:

Quality and relevance of document retrieval for downstream reasoning

Level	Indicators
-------	------------

edit, it's safer to stay in Protected View.

Enable Editing



Cognizant Skillspring

Excellent (12–15)	Relevant documents consistently retrieved; metadata and sources preserved
Good (8–11)	Retrieval works but relevance is inconsistent
Needs Improvement (0–7)	Poor or missing retrieval logic

4. Reasoning & Synthesis Quality (15 points)

What learners are evaluated on:

How well the system synthesizes information across documents.

Level	Indicators
Excellent (12–15)	Responses show clear cross-document reasoning and synthesis
Good (8–11)	Answers correct but mostly extractive or shallow
Needs Improvement (0–7)	Answers lack reasoning or coherence

5. Validation, Grounding & Guardrails (15 points)

What learners are evaluated on:

Trustworthiness and responsibility of the system.

Level	Indicators
Excellent (12–15)	Verifier agent enforces grounding, flags uncertainty, and applies guardrails
Good (8–11)	Basic validation present but not consistently applied
Needs Improvement (0–7)	No grounding or hallucination controls

6. Evaluation & Observability Mechanisms (10 points)

What learners are evaluated on:

How well the system evaluates itself and exposes decision-making.

Level	Indicators
Excellent (8–10)	Grounding checks, retrieval relevance, decision traces, and failure detection are logged and explained



Cognizant Skillspring

Good (5–7)	Partial evaluation signals available
Needs Improvement (0–4)	No meaningful evaluation or observability

7. Documentation & Explainability (10 points)

What learners are evaluated on:

Clarity of explanations provided to users and reviewers

Level	Indicators
Excellent (8–10)	Clear architecture diagrams, agent flows, and explanation of decisions
Good (5–7)	Documentation present but lacks depth
Needs Improvement (0–4)	Minimal or unclear documentation

8. Evaluation

1. **Document Creation:** Please create appropriate documents based on section 4 & 5.
2. Please find below Case Study Evaluation workflow

CaseStudy_Evaluation
n(OTJ)_Workflow_V1.J